

THEORY

1 - Define encryption and decryption. Determine why roles are needed in case of privileges. Explain with appropriate assumption.

Encryption and Decryption:

- Encryption is the process of converting plaintext into ciphertext to prevent unauthorized access. It uses algorithms and keys to encode data, ensuring that only authorized users can access the original content.
- Decryption is the reverse process of encryption. It converts ciphertext back into readable plaintext, using an appropriate decryption key.

Why Roles Are Needed in Case of Privileges:

Roles define the access level of different users to systems or data. In the context of encryption and decryption:

- Access Control: Certain roles (like admin, user, etc.) may have privileges to decrypt data, while others only have encryption access. This ensures sensitive data is only accessed by authorized individuals.
- Security: If a system allows all users to decrypt sensitive data, it becomes vulnerable to unauthorized access. Assigning roles prevents this by restricting decryption privileges.

Assumptions:

- Sensitive data should only be accessible to certain roles based on their responsibility (e.g., admin might be allowed to decrypt sensitive data, while a regular user might not).
- Decryption should only occur when the user has the appropriate key and privilege level.

In essence, roles in encryption and decryption ensure the data is secure and accessible only to the right people.

2 - Explain the difference between dependency preservation and lossless decomposition with examples.

Over. Here's the explanation:

Dependency Preservation vs Lossless Decomposition

1. Dependency Preservation:

- When decomposing a relation into smaller relations (for normalization), dependency preservation ensures that all functional dependencies (FDs) from the original relation are preserved in the decomposed relations.
- **Example:**
Let's say we have a relation $R(A, B, C)$ with functional dependencies $A \rightarrow B$ and $B \rightarrow C$.
If we decompose this into $R_1(A, B)$ and $R_2(B, C)$, both $A \rightarrow B$ and $B \rightarrow C$ are still valid in the decomposed relations, preserving the dependencies.

2. Lossless Decomposition:

- A decomposition is lossless if we can recreate the original relation from the decomposed relations without any loss of information.

- **Example:**

If we decompose a relation $R(A, B, C)$ into $R_1(A, B)$ and $R_2(B, C)$, it is lossless if the common attribute (here B) between the two relations is a **key** in at least one of the decomposed relations. In this case, the decomposition is lossless because we can join R_1 and R_2 back together without losing any information.

Key Difference:

- **Dependency Preservation** focuses on retaining all functional dependencies in the decomposed relations.
- **Lossless Decomposition** ensures that we can reconstruct the original relation without any data loss.

Both are important for normalization but may not always be achievable simultaneously.

3 - Compare and contrast dense and sparse indexing techniques. Provide scenarios where each would be more effective.

Dense vs. Sparse Indexing:

1. Dense Indexing:

- **Definition:** Every record in the data has an index entry.
- **Space:** Less efficient (requires more space).
- **Search Time:** Faster (direct access to every record).
- **Use Case:** Small/medium datasets, where fast search is critical (e.g., customer databases).

2. Sparse Indexing:

- **Definition:** Index entries for only some records, often the first of each block.
- **Space:** More efficient (requires less space).
- **Search Time:** Slower (needs to check multiple entries).
- **Use Case:** Large datasets, where space efficiency is key (e.g., large book indexes).

Comparison:

Aspect	Dense Indexing	Sparse Indexing
Space Efficiency	Less efficient	More efficient
Search Speed	Faster	Slower
Best for	Small/medium datasets	Large datasets

When to use:

- **Dense:** Fast access needed, small to medium datasets.
- **Sparse:** Large datasets, space is a priority.

4 - Discuss the role of clustered indexes in improving query performance. Provide examples.

Role of Clustered Indexes in Query Performance:

1. Improved Search Performance:

- Since data is physically ordered based on the clustered index key, range queries (such as finding all records between two dates) can be processed more quickly. The query can take advantage of the data's physical order, reducing the need for sorting operations.

2. Efficient Range Queries:

- Clustered indexes excel when queries involve a range of values, such as "find all employees with salaries between \$50,000 and \$70,000." Since the data is already sorted by the indexed field, the database can quickly identify the range of records without having to scan all rows.

3. Reduced I/O Operations:

- By keeping data sorted on disk, clustered indexes reduce the number of disk accesses required to retrieve a range of values, thus improving the efficiency of query execution.

4. Faster Joins:

- For join queries that involve fields indexed by a clustered index, the physical ordering of the data allows the system to quickly match rows between tables, reducing query execution time.

5. Optimizing Aggregations:

- For aggregation queries (like summing or averaging over a range of data), clustered indexes can reduce the time spent in sorting data, thus speeding up the query.

Example Scenarios:

• Example 1: A Customer Database

- If a **customer database** has a clustered index on the "customer_id," querying for a specific customer or performing range queries (e.g., "find customers who joined in 2020") will be efficient because the data is stored in the order of customer IDs. For range queries, like those on dates or IDs, the data is already grouped, making it faster to retrieve.

• Example 2: Employee Salary Data

- In an **employee database**, having a clustered index on the "salary" field will make it faster to execute queries like "find all employees with a salary over \$100,000." Since the salary data is physically ordered on the disk, the database can more quickly find the relevant records without performing additional sorting..

5 - Do you think conflict-serializability and view-serializability will be enough to make schedule recoverable and cascade-less? Justify your answer through an example.

Conflict-Serializability and View-Serializability in Ensuring Recoverability and Cascade-Less Schedules

Conflict-serializability and **view-serializability** are both important concepts in database transaction management. However, they alone cannot guarantee that a schedule will be **recoverable** and **cascade-less**. Let's break down why:

1. Conflict-Serializability:

- **Definition:** A schedule is conflict-serializable if its transactions can be reordered to form a serial schedule, where transactions do not conflict (i.e., no two transactions simultaneously access the same data in a conflicting way).
- **Role in Recoverability:** Conflict-serializability ensures that a schedule is equivalent to a serial schedule, but it does not necessarily address the issue of recoverability. **Recoverability** requires that if a transaction reads the data written by another transaction, it should not commit before the other transaction does. Conflict-serializability does not guarantee that a transaction follows this rule. A schedule could still allow for the possibility of a **dirty read** if the transaction reading data commits before the one writing the data, making it non-recoverable.

2. View-Serializability:

- **Definition:** A schedule is view-serializable if the transactions in the schedule can be ordered such that each transaction reads the same data values as they would in some serial execution of the transactions, and the final writes are consistent with some serial order.
- **Role in Recoverability:** View-serializability, while ensuring a consistent view of data across transactions, does not guarantee that the schedule is recoverable. A schedule could be view-serializable but still allow for non-recoverable situations. Specifically, it does not address the issue of **transaction dependencies** that are critical for recoverability, such as ensuring that a transaction commits only after all transactions it depends on have committed.

Cascade-Less Schedules:

- A **cascade-less** schedule is one where a transaction does not read data written by another uncommitted transaction, preventing the possibility of cascading rollbacks. Both **conflict-serializability** and **view-serializability** do not inherently ensure that a schedule is cascade-less. A schedule could still allow cascading aborts if transactions are allowed to read uncommitted data from other transactions.

Example to Illustrate:

Consider the following schedule:

Transaction Operation Data Item

T1	Read(A)	A
T2	Write(A)	A
T1	Commit	
T2	Commit	

- **Conflict-serializability:** The schedule is conflict-serializable because we can reorder it to form a serial schedule where T1 runs before T2. However, this schedule is **not recoverable** because T1 reads A before T2 writes it, and T1 commits before T2. If T2 aborts, T1 would have already committed, violating the **recoverability** condition.

- **View-serializability:** The schedule is also view-serializable because the reads and writes form a consistent view. However, it does not guarantee **cascade-less** behavior, as T1 reads data written by T2, leading to potential cascading rollbacks if T2 aborts.

6 - Explain how a transaction will be accomplished through different states. Also, illustrate which properties are mandatory in terms of database transactions.

How a Transaction is Accomplished through Different States

A **database transaction** is a sequence of operations performed as a single unit of work. It goes through different states during its lifecycle. Here's how it works:

1. **Initial State:**
 - The transaction begins when the user requests it. In this state, the transaction is **active**, and it starts executing operations on the database.
2. **Execution (Active State):**
 - The transaction performs its operations, which could include reading or writing data in the database.
3. **Commit State:**
 - If the transaction completes successfully, the transaction enters the **commit state**. Here, all the changes made during the transaction are **permanently saved** to the database.
4. **Abort/Failure State:**
 - If there is an error or the transaction violates constraints, the transaction enters the **abort state**. In this case, any changes made by the transaction are rolled back, and the database remains unchanged.
5. **Final State (Complete):**
 - The transaction is either committed (changes are permanent) or aborted (changes are discarded), and the system is left in a consistent state.

Mandatory Properties of Database Transactions (ACID Properties)

1. **Atomicity:**
 - **Definition:** A transaction is atomic, meaning it is treated as a single unit. It either completes entirely or doesn't execute at all. If a failure occurs during the transaction, all changes are rolled back.
 - **Example:** If a transfer of money from Account A to Account B is part of the transaction, both withdrawal and deposit must succeed together. If one fails, the entire transaction fails.
2. **Consistency:**
 - **Definition:** The transaction must transition the database from one valid state to another. The database should remain in a consistent state before and after the transaction.
 - **Example:** If a transaction violates database constraints (like trying to insert invalid data), the transaction is aborted to maintain consistency.
3. **Isolation:**

- **Definition:** Each transaction must execute independently of other transactions. Intermediate results of a transaction should not be visible to other transactions until it is completed.
- **Example:** Two transactions updating the same account should not interfere with each other. If one transaction updates a record, the other must wait until the first transaction is complete.

4. Durability:

- **Definition:** Once a transaction is committed, the changes are permanent, even in the case of a system failure.
- **Example:** If the system crashes after a transaction is committed, the changes (like a successful transfer) will still be in the database after recovery.

Importance of Distributed Databases

A **distributed database** is a collection of databases distributed across multiple locations but functions as a single system. The importance of distributed databases includes:

1. Fault Tolerance:

- Distributed databases are more resilient to failures. If one node fails, other nodes can still function, ensuring system availability.

2. Scalability:

- Distributed databases can easily scale by adding more nodes to handle increasing data and transaction loads, ensuring efficient management of large datasets.

3. Data Availability:

- By storing copies of the data in different locations, distributed databases ensure higher availability, even during network failures or site outages.

4. Load Balancing:

- With data spread across multiple locations, distributed databases can balance the workload and provide better performance by spreading queries and transactions among different nodes.

5. Improved Performance:

- They enable faster data access by storing data closer to where it's needed, reducing latency.

In conclusion, **ACID properties** (Atomicity, Consistency, Isolation, and Durability) are mandatory for ensuring reliable transactions in a database, and **distributed databases** offer benefits like fault tolerance, scalability, and improved performance by distributing data across multiple sites.

7 - Discuss the importance of distributed databases in modern systems. Explain the trade-offs between replication and fragmentation.

Importance of Distributed Databases in Modern Systems

Distributed databases are used widely in modern systems for the following reasons:

1. Scalability:

- They can easily grow by adding more servers or nodes to handle more data, which makes them suitable for businesses with increasing data needs.

2. High Availability:

- Since data is stored in multiple places, the system keeps working even if one server fails. This ensures that the system is always up and running.

3. Fault Tolerance:

- Distributed databases keep copies of data across multiple nodes, meaning if one part fails, another can take over, preventing downtime.

4. Improved Performance:

- Data can be placed closer to users, reducing delays. Also, multiple servers can share the workload, improving speed.

5. Geographic Distribution:

- Data can be stored in different locations around the world, which helps meet laws about where data can be stored and makes access faster for users in different regions.

Trade-offs Between Replication and Fragmentation

Replication and Fragmentation are two ways of managing data in distributed databases. Here's how they differ:

1. Replication:

- **What it is:** Keeping copies of the same data on multiple servers.

Advantages:

- **High Availability:** Data is still available if one server goes down.
- **Better Read Speed:** Multiple copies can help handle many read requests at once.

Disadvantages:

- **Slower Writes:** When data changes, all copies need to be updated, which can slow things down.
- **More Storage:** Storing multiple copies uses more space.

2. Fragmentation:

- **What it is:** Dividing data into smaller pieces (fragments) and storing them on different servers.

Advantages:

- **Faster Queries:** If the data is close to where it's needed, queries run faster.
- **Efficient Storage:** Storing smaller pieces is more space-efficient.

Disadvantages:

- **Complex Queries:** If data is spread out, it can be harder and slower to gather data from multiple fragments.
- **Consistency:** Keeping fragments in sync can be tricky, especially with frequent updates.

8 - What is the role of logging in database recovery?

Role of Logging in Database Recovery

Logging is essential for recovering a database after a failure (like a crash). Here's how it works:

1. Keeping Track of Changes:

- Logging records all changes made to the database (e.g., adding, updating, or deleting data) in a log file.
- These records include details like the transaction ID and the old and new values of the data.

2. Ensuring Data is Not Lost (Durability):

- If a crash happens before changes are saved to the database, the log can be used to recover the changes.
- This ensures that once a transaction is completed, its changes are not lost, even if the system crashes.

3. Recovering After a Crash:

- When the system restarts after a crash, the log is checked to see which transactions were completed or incomplete:
 - Redo: If a transaction was completed but not written to the database, the log will apply those changes again.
 - Undo: If a transaction was not completed, the log will undo the changes it made.

4. Helping with Consistency:

- The log ensures that the database is always in a consistent state after a crash, meaning it's never left in an incomplete or incorrect state.

9 - Discuss two techniques for resolving system failures in a database system.

Two Techniques for Resolving System Failures in a Database

When a database system crashes or faces a failure, two important techniques help recover and restore the database:

1. Checkpointing

What it is:

- **Checkpointing** is when the database saves its current state to disk at certain intervals. It marks a point where everything in the system is safe and stable.

How it works:

- At regular times, the database records a **checkpoint** with all the data up to that point.
- If the system crashes, it can start from the last checkpoint, so only changes after that need to be checked or corrected.

Why it's useful:

- **Faster recovery:** After a crash, recovery starts from the checkpoint, saving time by not having to go through every transaction.

2. Logging

What it is:

- **Logging** records every change or action made to the database. These logs are saved before any actual data changes happen.

How it works:

- Each transaction's changes are written into a log file (like a diary of database actions).
- If the system crashes, the log can be used to either **undo** unfinished transactions or **redo** completed ones to ensure everything is consistent.

Why it's useful:

- **Data recovery:** Logs help fix any changes that were not saved properly, ensuring that the database ends up in a good state.
-

10 - Why data normalization is important? Explain lossless decomposition and lossy decomposition with an example.

Why Data Normalization is Important

Data normalization is a process in database design that organizes data to reduce redundancy and improve data integrity. It helps to structure the database in such a way that it meets the following goals:

1. **Minimizes Data Redundancy:** Normalization eliminates duplicate data, which saves storage space and ensures that the data is consistent.
2. **Improves Data Integrity:** By reducing redundancy, normalization makes it easier to maintain consistency in data, ensuring there are fewer chances of errors or contradictions.
3. **Simplifies Database Maintenance:** When data is well-organized and normalized, it is easier to update, delete, or add new information without causing inconsistencies or requiring major changes.
4. **Optimizes Query Performance:** Normalization can sometimes improve query performance because the data is structured and organized better, reducing the need to check for duplicates.

Lossless Decomposition

Definition:

- **Lossless decomposition** refers to splitting a table into smaller tables in such a way that no information is lost, and you can still reconstruct the original table using these smaller tables.

Key Point:

- After decomposition, you should be able to **join** the smaller tables back together without losing any data.

Example:

Let's say we have a table for **Student** and **Course** information:

Student_ID	Name	Course	Instructor
1	Alice	Math	Dr. Smith

Student_ID	Name	Course	Instructor
2	Bob	History	Dr. Jones
1	Alice	History	Dr. Jones
3	Charlie	Math	Dr. Smith

We decompose the table into two smaller tables:

Student Table:

Student_ID	Name
1	Alice
2	Bob
3	Charlie

Course Table:

Student_ID	Course	Instructor
1	Math	Dr. Smith
2	History	Dr. Jones
1	History	Dr. Jones
3	Math	Dr. Smith

By using **JOIN** between the two tables on Student_ID, we can **reconstruct** the original table without losing any data. This is an example of **lossless decomposition**.

Lossy Decomposition

Definition:

- **Lossy decomposition** happens when we split a table into smaller tables, and after joining them back, some information is lost or not recoverable.

Key Point:

- After decomposition, the original table cannot be fully reconstructed by simply joining the smaller tables, meaning some data is lost.

Example:

Using the same **Student** and **Course** table, suppose we decompose the table as follows:

Student Table:

Student_ID	Name
1	Alice
2	Bob

Course Table:

Course Instructor

Math Dr. Smith

History Dr. Jones

In this case, if we try to join these two tables, we cannot recover the original data about which student took which course because we have lost the relationship between students and courses. This decomposition is **lossy** because some details (like Alice taking Math) are lost and can't be recovered by joining the tables.

11 – How to choose indexing or hashing technique? Justify.

Choosing Between Indexing and Hashing

Both **indexing** and **hashing** are techniques used to speed up data retrieval, but they are suitable for different types of queries. Here's a simple breakdown of when to use each:

1. Indexing:

- **Best for:** When you need to **search** or **sort** data in **ranges** (e.g., finding all students who scored between 50 and 80).
- **How it works:** It creates an ordered list or tree structure, so you can quickly search, sort, or filter data.
- **Use it when:**
 - You need **range queries** (like "find records between a certain range").
 - You want to **sort** data or perform queries with **multiple conditions** (e.g., find students with a certain grade and age).
- **Example:** Searching for employees with salaries between \$50,000 and \$80,000.

Pros:

- Works well with **sorted** data.
- Good for **range-based** searches.

Cons:

- Takes up extra **storage**.
- **Slower writes** because the index must be updated whenever data changes.

2. Hashing:

- **Best for:** When you need to find data based on a **specific key** quickly (e.g., finding a record by employee ID).
- **How it works:** It uses a function to map the data to a specific "location" (address), making it super fast for exact matches.
- **Use it when:**
 - You need **exact matches** (e.g., look up a student by student ID).
 - You don't need **range queries**.
- **Example:** Searching for a student by their unique ID.

Pros:

- **Very fast** for exact lookups.
- **Efficient storage**.

Cons:

- **No range queries** (you can't easily search for a range of values).
 - **Collisions** (two keys might point to the same location, causing slowdowns).
-

When to Choose Which?

- **Choose Indexing** if you need to:
 - Search by a **range** of values or **order**.
 - Do **complex searches** with multiple conditions.
- **Choose Hashing** if you need:
 - **Fast lookups** for specific records (like an ID).
 - **Exact matches** and don't need range queries.

12 - Explain the purpose of checkpoint mechanism. How often should checkpoints be performed give an example.

Purpose of Checkpoint Mechanism

The **checkpoint mechanism** in a database system is used to **save the current state of the database** at regular intervals to stable storage (e.g., disk). This allows the database to recover more quickly in case of a system failure (such as a crash or power outage). Here's why it is important:

1. Reduces Recovery Time:

- When a system crashes, the database needs to restore data to a consistent state. Checkpoints minimize the amount of data that needs to be checked or recovered by marking a point in time when the system was consistent and stable.

2. Improves Efficiency:

- Without checkpoints, the system would have to redo all transactions since the last crash, which could be time-consuming. By creating checkpoints, only the transactions after the last checkpoint need to be reprocessed.

3. Consistency:

- Checkpoints ensure that all data modifications up to a certain point are safely written to the disk, making sure that after recovery, the database remains in a consistent state.
-

How Often Should Checkpoints Be Performed?

The frequency of checkpoints depends on various factors like transaction volume, system performance, and the amount of time you're willing to spend on recovery. Here are key factors to consider:

1. Transaction Volume:

- In a system with **high transaction rates**, checkpoints should be performed **more frequently** to ensure the system doesn't have to redo too many transactions during recovery.

2. Recovery Time Objective (RTO):

- If you need **fast recovery**, checkpoints should be performed often, so only a small number of transactions need to be recovered after a crash.

3. Performance Impact:

- Too frequent checkpoints can cause **performance overhead** (due to disk writes), so checkpoints should be spaced out to avoid slowing down the system too much.

Example of Checkpointing:

Imagine a database processing 1,000 transactions per minute. If a checkpoint occurs every **10 minutes**, and the system crashes after 9 minutes, only the last 100 transactions need to be redone. This reduces recovery time compared to not having checkpoints, where the database would need to go back and redo all 1,000 transactions.

Balance: The checkpoint interval should be chosen to balance between recovery speed (too long means more data to recover) and system performance (too frequent means more overhead).

12 - Describe Encryption Decryption and Privilege .

Encryption, Decryption, and Privileges

Let's break down **encryption**, **decryption**, and **privileges** in simple terms:

1. Encryption

What it is:

- **Encryption** is the process of converting **plain, readable data** (called **plaintext**) into an unreadable format (called **ciphertext**) using a specific algorithm and a key. This ensures that the data is protected from unauthorized access.

Why it's used:

- It keeps sensitive information secure, especially when transmitted over networks or stored on servers.
- Only authorized users with the proper key can access the original data.

Example:

- You send a password over the internet, and it is encrypted to keep it safe from hackers.

2. Decryption

What it is:

- **Decryption** is the process of converting **ciphertext** (encrypted data) back into its original, readable format (**plaintext**).

How it works:

- Decryption uses a **key** (often the same key used for encryption in **symmetric encryption** or a different one in **asymmetric encryption**) to reverse the encryption process.

Example:

- The recipient of the encrypted message uses the decryption key to get back the original message.

3. Privileges

What they are:

- **Privileges** refer to the **permissions** granted to users or roles to access and perform operations on a database.

Types of privileges:

- **Read:** Allows the user to view data.
- **Write:** Allows the user to modify data.
- **Execute:** Allows the user to run certain functions or procedures.
- **Delete:** Allows the user to remove data.

Why they are important:

- **Privileges** help maintain **security** by controlling who can access or modify certain data.
- It ensures that only **authorized users** can perform sensitive operations, keeping the system safe.

Example:

- A user might have permission to **view** certain data in a database but might not have permission to **delete** it.

